

Guía rápida para trabajar con tu proyecto

Venv, Jupyter, VS Code y Git desde terminal

Enfoque	Repositorio local + entorno virtual + notebooks SQL con ipython-sql
Uso ideal	Referencia práctica para arrancar rápido y trabajar con orden

Idea general: El flujo es: clonar repo -> crear venv -> activarlo -> instalar requirements -> instalar Jupyter/ipython-sql -> abrir VS Code -> crear rama -> trabajar -> commit -> push.

1. Flujo recomendado de trabajo

- Clona tu repositorio en una carpeta local.
- Crea un entorno virtual dentro del proyecto.
- Activa el entorno virtual antes de instalar dependencias.
- Instala lo que venga en `requirements.txt`.
- Si vas a usar notebooks con SQL, instala `jupyter`, `ipykernel`, `ipython-sql` y `pymysql`.
- Abre el proyecto en VS Code y selecciona el intérprete del `venv`.
- Crea una rama nueva para trabajar con cambios aislados.
- Haz commits pequeños y descriptivos.
- Sube tu rama al remoto con `push`.

2. Preparar el proyecto desde cero

2.1 Clonar el repositorio

```
git clone https://github.com/tu-usuario/tu-repo.git
cd tu-repo
```

2.2 Crear el venv

```
python -m venv venv
```

Alternativa en Windows: Si `python` no responde bien, prueba con: `py -m venv venv`

2.3 Activar el venv

PowerShell

```
.\venv\Scripts\Activate.ps1
```

CMD

```
venv\Scripts\activate
```

Si PowerShell bloquea la activación, ejecuta esto una vez en esa sesión:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

2.4 Instalar dependencias del repo

```
pip install -r requirements.txt
```

2.5 Instalar extras para notebooks SQL

```
pip install jupyter ipykernel ipython-sql pymysql sqlalchemy
```

Nota: `sqlalchemy` no siempre es obligatorio, pero suele ser útil. Para `ipython-sql` con MySQL, `pymysql` sí te conviene tenerlo.

2.6 Registrar el kernel del entorno

```
python -m ipykernel install --user --name=sql_repo_env --display-name "Python (sql_repo_env)"
```

2.7 Abrir el proyecto en VS Code

```
code .
```

3. Conexión SQL en cada notebook con ipython-sql

Usa siempre un bloque inicial simple. Así evitas errores y puedes reutilizarlo en todos tus notebooks.

```
%load_ext sql
from urllib.parse import quote_plus

user = "root"
password = quote_plus("tu_password")
host = "localhost"
port = "3306"
db = "sakila"

conn = f"mysql+pymysql://{user}:{password}@{host}:{port}/{db}"
%sql $conn
```

Consulta de prueba:

```
%%sql
SELECT *
FROM actor
LIMIT 5;
```

Importante: Si tu contraseña tiene caracteres como @, :, / o espacios, quote_plus ayuda a que la URL de conexión no falle.

4. Qué revisar en VS Code

- Abre la paleta con Ctrl + Shift + P.
- Usa Python: Select Interpreter.
- Elige el Python que apunta a .\venv\Scripts\python.exe.
- Al abrir un .ipynb, revisa el kernel en la parte superior derecha.
- El kernel ideal debe ser el que registraste con ipykernel.

5. Git desde terminal: lo básico

5.1 Ver estado del repo

```
git status
```

5.2 Ver en qué rama estás

```
git branch
```

5.3 Traer últimos cambios

```
git pull
```

5.4 Agregar archivos al stage

```
git add .
```

5.5 Crear un commit

```
git commit -m "Agrego notebooks SQL de practica"
```

5.6 Subir cambios

```
git push
```

6. Cómo crear y usar una rama

Trabajar en ramas te permite hacer cambios sin tocar directamente main o master.

6.1 Crear una rama nueva y cambiarte a ella

```
git checkout -b feature/sql-notebooks
```

6.2 Ver tus ramas

```
git branch
```

6.3 Subir la rama al remoto

```
git push -u origin feature/sql-notebooks
```

6.4 Cambiar de rama

```
git checkout main  
git checkout feature/sql-notebooks
```

6.5 Actualizar tu rama con cambios recientes de main

```
git checkout main  
git pull  
git checkout feature/sql-notebooks  
git merge main
```

Convención sugerida: Usa nombres simples y claros: feature/sql-notebooks, fix/venv-setup, chore/readme.

7. Flujo diario sugerido con Git

```
git checkout main  
git pull  
git checkout -b feature/mi-cambio  
  
# trabajas en tus archivos  
  
git status  
git add .  
git commit -m "Descripcion corta y clara"  
git push -u origin feature/mi-cambio
```

8. Comandos útiles extra

Comando	Uso
git log --oneline	Ver historial resumido de commits
git diff	Ver cambios no confirmados
git restore archivo.ext	Descartar cambios en un archivo
git rm --cached archivo.ext	Quitar un archivo del stage sin borrarlo localmente
pip list	Ver paquetes instalados en el venv
where python	Ver qué Python estás usando en Windows

9. .gitignore recomendado

```
venv/  
.ipynb_checkpoints/  
__pycache__/  
.env
```

10. Problemas comunes

El venv no activa: Revisa si estás en la carpeta correcta y si debes usar PowerShell o CMD. En PowerShell, quizá necesites Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass.

VS Code usa otro Python: Selecciona manualmente el intérprete del venv en la paleta de comandos.

El notebook no reconoce %sql: Instala ipython-sql dentro del mismo venv y reinicia el kernel.

La conexión a MySQL falla: Verifica usuario, contraseña, puerto, servicio de MySQL y el nombre de la base.

Git rechaza el push: Haz git pull antes y revisa si hay cambios remotos que debas integrar.

11. Chuleta final

```
git clone <repo>  
cd <repo>  
python -m venv venv  
.\venv\Scripts\Activate.ps1  
pip install -r requirements.txt  
pip install jupyter ipykernel ipython-sql pymysql sqlalchemy  
python -m ipykernel install --user --name=sql_repo_env --display-name "Python (sql_repo_env)"  
code .
```